

NOA-LSTM: An efficient LSTM cell architecture for time series forecasting

Hemant Yadav^{a,*}, Amit Thakkar^b

^a Smt. Kundanben Dinsha Patel Department of Information Technology, Chandubhai S. Patel Institute of Technology, Faculty of Technology, Charotar University of Science and Technology, India

^b Department of Computer Science and Engineering, Chandubhai S. Patel Institute of Technology, Faculty of Technology, Charotar University of Science and Technology, India

ARTICLE INFO

Keywords:

Time series forecasting
Artificial neural network (ANN)
Recurrent neural network (RNN)
Long short term memory network (LSTM)
Deep learning

ABSTRACT

The application of Machine learning and deep learning techniques for time series forecasting has gained significant attention in recent years. Numerous endeavors have been devoted to automating forecasting through the utilization of cutting-edge neural networks. Notably, the recurrent neural network (LSTM – Long Short-Term Memory) has emerged as a central concept in most research endeavors. Although LSTM was initially introduced in 1997 for sequence modeling, subsequent updates have primarily focused on language learning tasks. These updates have introduced various computational mechanisms within the LSTM cell, including the forget gate, input gate, and output gate. In this study, we investigate the impact of each computational component in isolation to analyze their effects on time series forecasting tasks. Our experiments utilize the Jena weather dataset and Appliance Energy Usage time series for evaluation. The experimental results reveal that variations of the LSTM model outperform the most popular LSRM cell format in terms of error rate and training time. Specifically, the variations identified in this paper demonstrate superior generalization capabilities and yield reduced forecasting errors.

1. Introduction

Time series forecasting (TSF) is a crucial aspect in addressing various practical issues that involve a temporal aspect. In various domains such as wind power (Zhao et al., 2016), building energy usage (Deb et al., 2017), financial time series (Cheng et al., 2022), retail industries (Ensafi et al., 2022), Healthcare (Fernández-Martínez & Jaramillo-Morán, 2022), traffic forecasting (Yu, 2022), etc using TSF to predict the future is an important research issue. The chronological order of observation makes time series data difficult to analyze. Therefore, over the last decades, researchers have been interested in developing state-of-the-art models or methods that can be used to capture the non-linear relationship of time series data. the utilization of machine learning (ML) and deep learning (DL) techniques has been surging in popularity for numerous data-driven applications and challenges, including time series forecasting (TSF) as evidenced (Masini et al., 2021). Deep neural network (DNN) is a superior alternative to statistical forecasting methods since DNNs can be used to estimate multi-target time series (Reyes & Ventura, 2019). Moreover, the capacity of DNNs to adjust to data without any preconceived assumptions offers considerable benefits when working with real-time data streams of time series. (Lara-Benítez

et al., 2020).

Neural Networks specifically deep neural networks are known for their ability to learn any non-linear function approximations (Nielsen, 2015). In the paper (Ömer Faruk, 2010) a feed-forward neural network was used to model the non-linear relationships based on residues of the sequential data with the fact that the time series data comprises both linear and non-linear patterns. To reduce the computational burden of handling linear and nonlinear learning separately, the use of recurrent neural networks (RNNs) was proposed as RNNs are designed for modeling sequential data (Hewamalage et al., 2021). However, as indicated by (Y. Yu et al., 2019), RNNs are having problems with vanishing gradients or exploding gradients which require special attention to the design learning process during backpropagation using gradient descent. Enhanced versions of the RNN block, the long short-term memory (LSTM) networks ensure to capture of sequential dependencies more effectively and are used as building blocks of the encoder-decoder networks for time series forecasting tasks (Van Houdt et al., 2020). Apart from capturing temporal relations effectively, LSTMs have special mechanisms to deal with vanishing gradients and exploding gradients which makes them a candidate for developing robust time series forecasting models (Alom et al., 2019). There are many state-of-

* Corresponding author.

E-mail addresses: hemantnyadav.it@charusat.ac.in, hemantnyadav@gmail.com (H. Yadav), amitthakkar.it@charusat.ac.in (A. Thakkar).

<https://doi.org/10.1016/j.eswa.2023.122333>

Received 6 July 2023; Received in revised form 14 October 2023; Accepted 23 October 2023

Available online 28 October 2023

0957-4174/© 2023 Elsevier Ltd. All rights reserved.

Table 1

Comparison of literature in which LSTM used in its vanilla form.

Publication Year	Dataset	LSTM Variations	Key Findings/Idea
2015 (Chen et al., 2015)	Stock Prices	Vanilla LSTM	LSTM outperforms traditional models (ARIMA) for stock price prediction
2015 (Tian & Pan, 2015)	Traffic Flow	Stacked LSTM	Stacked LSTM is used to determine the optimal time lags dynamically which results in higher generalization ability.
2018 (Hewage et al., 2021)	Weather Data	Bidirectional LSTM (Bi-LSTM)	Bi-LSTM captures complex patterns in weather data
2019 (Pan et al., 2019)	Solar generation	LSTM with attention	The estimation capability of LSTM was improved by using an attention mechanism.
2020 (Bandara et al., 2020)	CIF2016 and NN5competitions	Clustering + LSTM	Clustering-related time series improves training time and reduces forecasting errors significantly.
2022 (Ng et al., 2022)	Social Media Data	LSTM with exogenous variables	Estimation of the daily volume of topic-specific social media activities is proposed by incorporating multiple exogenous variables.
2022 (Lim et al., 2022)	Solar Power Data	CNN + LSTM	Classification of weather patterns using CNN to improve the forecasting accuracy of LSTM

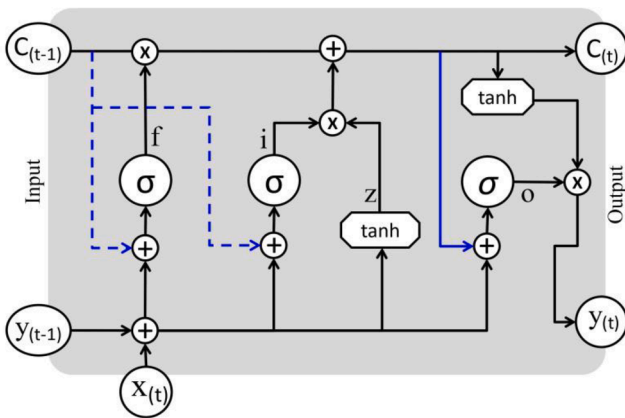


Fig. 1. LSTM with peephole connection: In this figure, σ is a sigmoid operation, input is indicated by $X(t)$ at time t and output is indicated using $Y(t)$ at time t . $C(t)$ represents cell state/memory while the addition and multiplication operations are represented using $+$ and $\times$ symbols.

the-art methods for time series forecasting which use LSTM cells in some combination or by some special modifications such as DeepAR (Salinas et al., 2020) which is a probabilistic forecast using autoregressive recurrent neural network, Deep State Space (Rangapuram et al., n.d.), Deep Renewal Processes (Türkmen et al., 2021), Deep Factor (Wang et al., n.d.). Apart from machine learning methods there are some state-of-the-art statistical methods such as Facebook Prophet used in for covid 19 data analysis and predictions (Dash et al., 2021).

However, most of the time, these newly developed neural net techniques are not tested on common ground or not thoroughly tested and

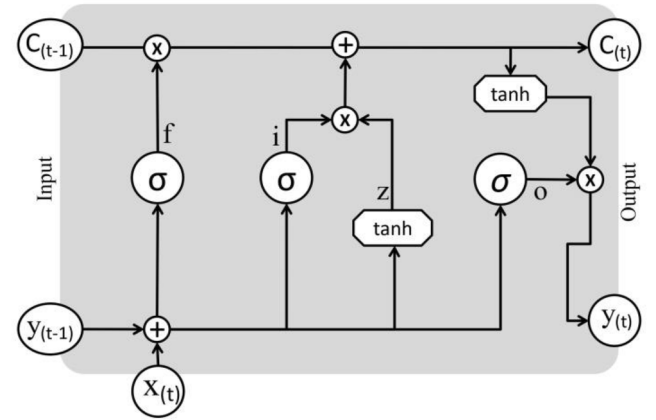


Fig. 2. Base LSTM.

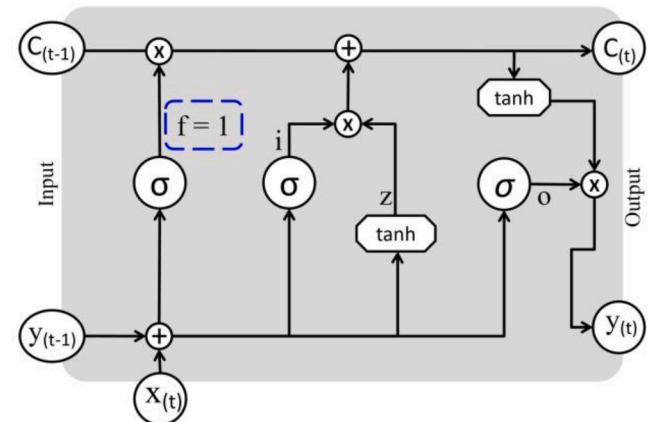


Fig. 3. LSTM without forget gate NFG.

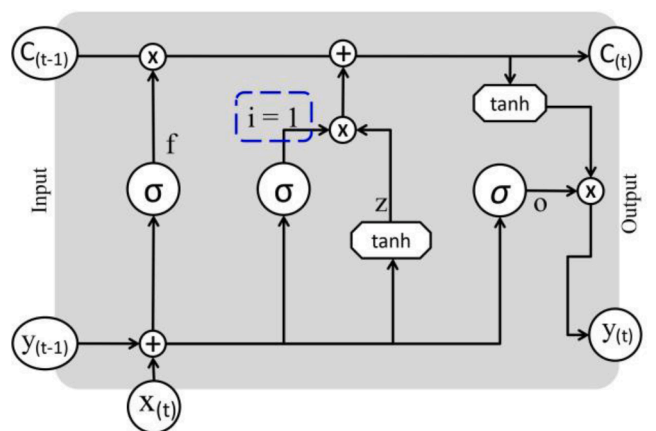


Fig. 4. LSTM without output gate NOG.

experimented to be used in practice (Shewalkar et al., 2019). The LSTM architecture has been criticized for being ad-hoc and having many variable components whose contributions are not evident. Consequently, it is uncertain whether the popular LSTM cell is the most optimal architecture or if there is a possibility that superior architectures may exist. All literature indicates requirements for a study of the effects of different computational components in the LSTMs targeting time series forecasting. In Table 1 list of different approaches over the period using LSTM for time series forecasting is given but all of them are lacking

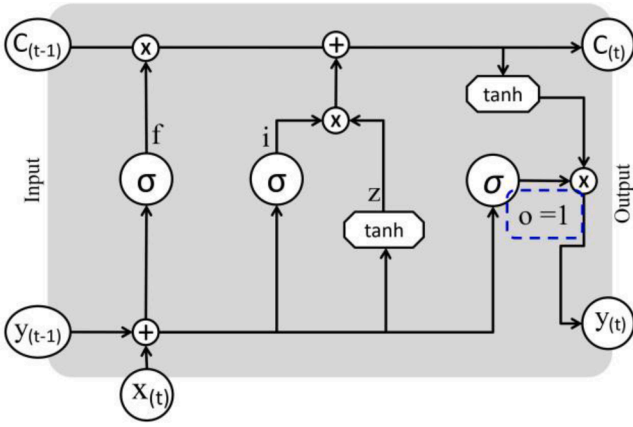


Fig. 5. LSTM without output gate NOG.

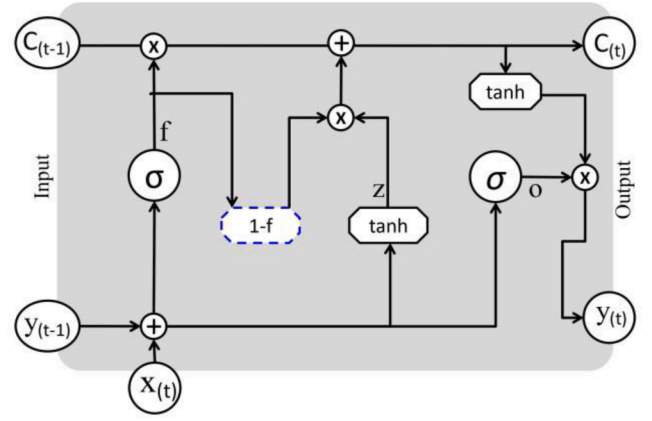


Fig. 8. LSTM without coupled input and forget gate CIFG.

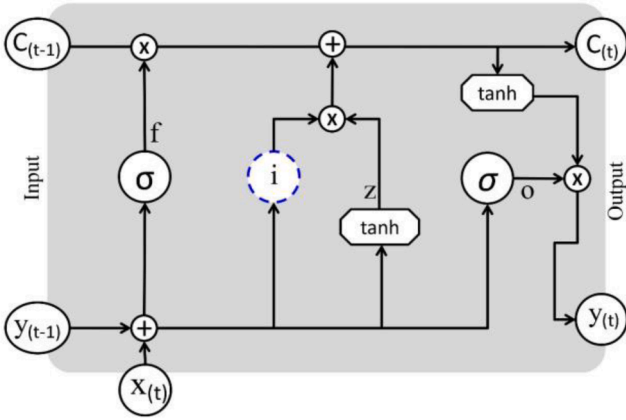


Fig. 6. LSTM without input activation function NIA.

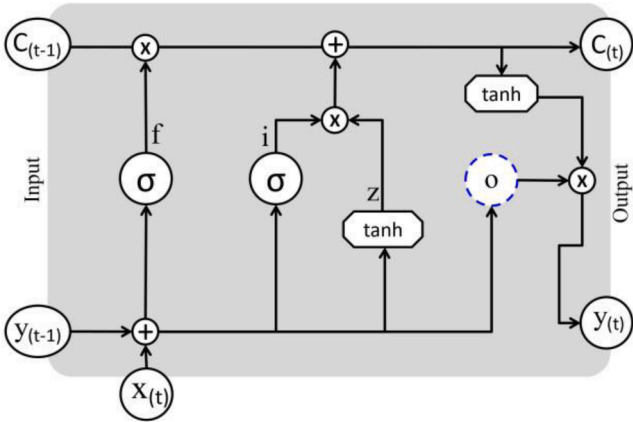


Fig. 7. LSTM without output activation function NOA.

Table 2

Dataset Details.

Dataset Name	Target Variable	Target Variable Range	Length of Time Series	Sample Rate
Jena Climate Dataset	Temperature (Degree C)	Max: 37.28 Min: -22.76	70,091	10 Min
Appliance Energy Complete	Appliance Energy Usage (kWh)	Max: 1080.00Min: 10.0	19,735	10 Min

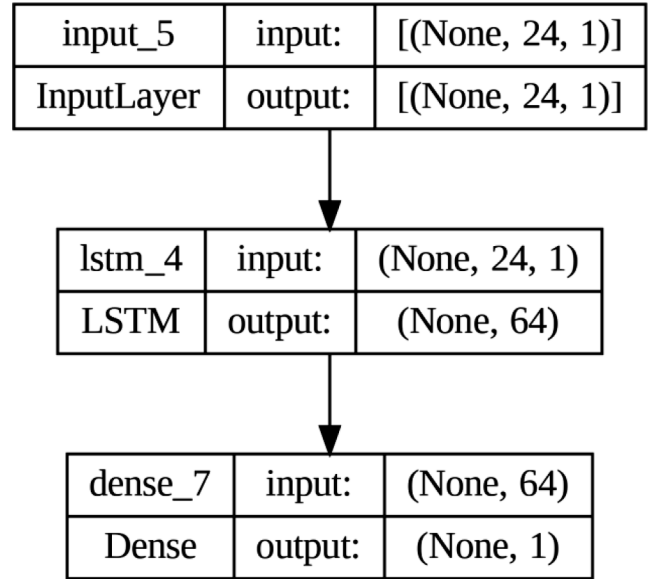


Fig. 9. Neural Net Architecture.

in discussing the effect of variable components such as the gating mechanism in LSTM.

This paper aims to address this gap by conducting an empirical evaluation of the effect of each computational element of the LSTM cell architecture. In this study, LSTM and GRU (Gated Recurrent Unit - which is one of the most popular RNN cells after LSTM (Shewalkar et al., 2019)) were compared against six different variants created by removing exactly one element of LSTM to check the effect of each element individually. This paper discusses the background of the most

used LSTM architecture in Section 2 and details of how the variations in LSTM are created in Section 3. Experimental details are discussed in Section 4. Section 5 is devoted to results and discussion and finally, Section 6 is about the conclusion and future directions.

2. Background of LSTM

Recurrent Neural Networks (RNNs) are a type of neural network in which each neuron receives input from both its previous output and the current timestamp's input. The distinctive characteristics of RNNs primarily arise from the choice of recurrent cell and network architecture.

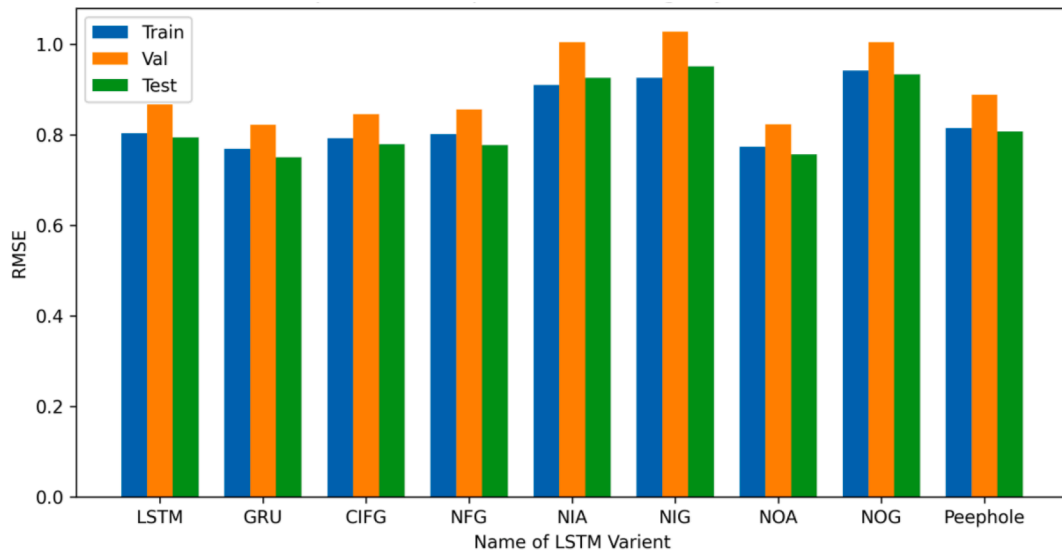


Fig. 10. RMSE on Jena Climate Dataset for Train, Test and Validation data.

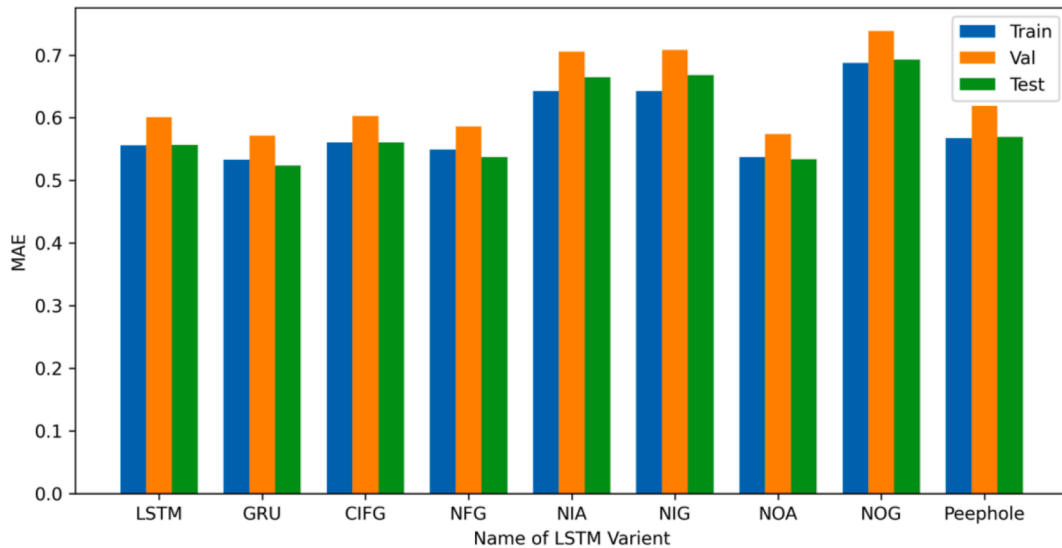


Fig. 11. MAE on Jena Climate Dataset for Train data, Validation data, and Test data.

Table 3

RMSE and MAE for Train, validation, and test datasets in sorted order for the Jena Climate Dataset.

Model Name	Train Dataset		Validation Data		Test Data	
	RMSE	MAE	RMSE	MAE	RMSE	MAE
GRU	0.770	0.533	0.823	0.572	0.750	0.524
NOA	0.774	0.537	0.823	0.574	0.757	0.534
NFG	0.802	0.549	0.857	0.586	0.778	0.537
LSTM	0.804	0.556	0.868	0.601	0.795	0.556
CIFG	0.792	0.561	0.846	0.603	0.780	0.561
PeepholeLSTMCell	0.815	0.568	0.889	0.619	0.808	0.570
NIG	0.926	0.643	1.005	0.706	0.926	0.665
NIA	0.910	0.643	1.028	0.708	0.952	0.668
NOG	0.943	0.687	1.005	0.739	0.934	0.693

LSTM (Hochreiter & Schmidhuber, 1997) probably was considered the most suitable cell to capture long and short-term temporal relationships in sequence data and resolve vanishing gradient problems. This fundamental recurrent (LSTM) unit has undergone several modifications throughout time to overcome its flaws or to apply according to

domain requirements.

The forget gate, which enables the LSTM to reset its state, was initially proposed in the first work to suggest a change in the LSTM design (Gers et al., 2000). Once more (Gers et al., n.d.), it was claimed that the cell must regulate the gates to learn accurate timings. Peephole connections were introduced to the architecture to enable learning accurate timings (blue lines from cell state to gating mechanism shown in Fig. 1). The last adjustment to the original LSTM was given by (Graves & Schmidhuber, 2005) by introducing the training of the LSTM network by full back-propagation through time (BPTT).

A variant of the LSTM architecture proposed by (Cho et al., 2014) known as Gated Recurrent Unit (GRU) is simplified by combining the forget and input gate making a new gate called the update gate without using peepholes or output activation mechanisms. Finally, only the recurring connections to the input block are gated by their output gate (called the reset gate). A comparative study of LSTM and GRU was published by (Chung et al., 2014) but the results reported mostly did not have a clear opinion. The most frequently utilized LSTM configuration in applications was first introduced by (Graves & Schmidhuber, 2005). It is referred to as “vanilla LSTM” and is the standard against which all other

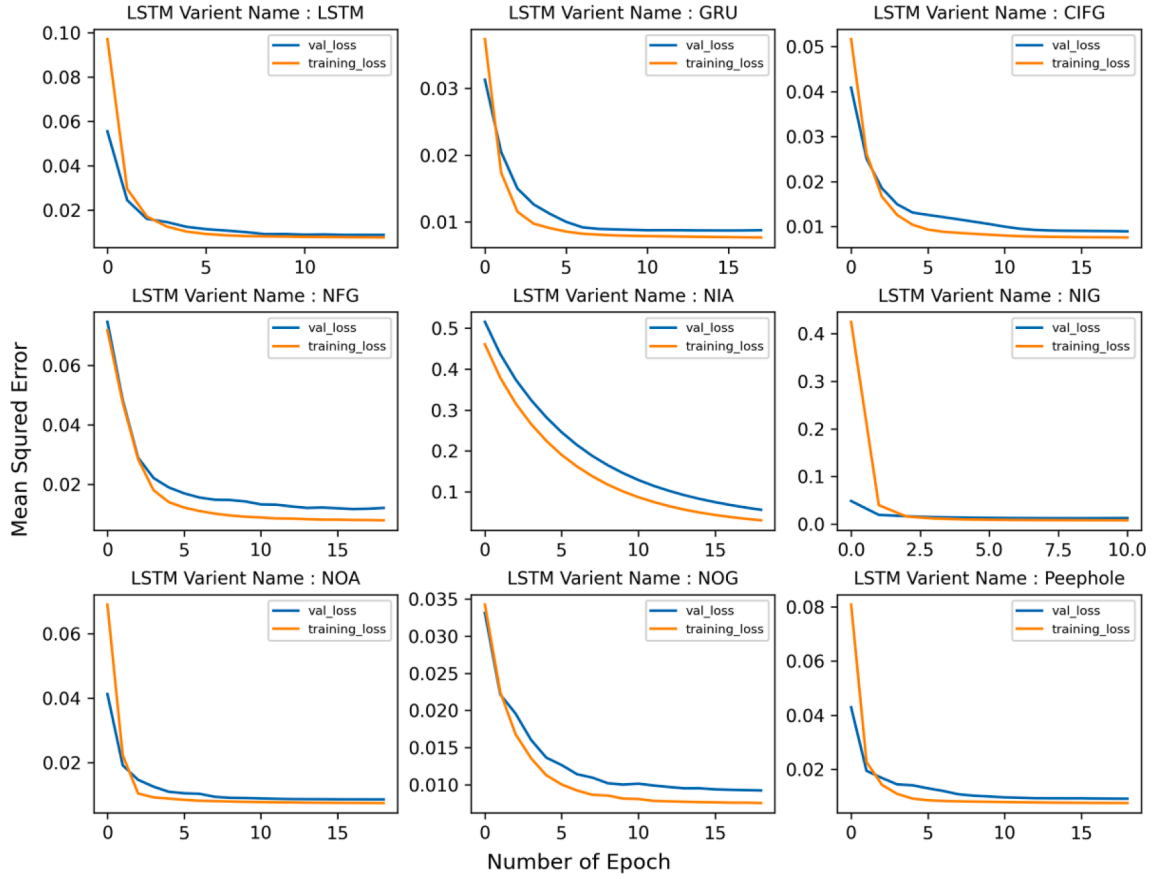


Fig. 12. Validation loss and Training Loss for each Variant of the Jena Climate Dataset.

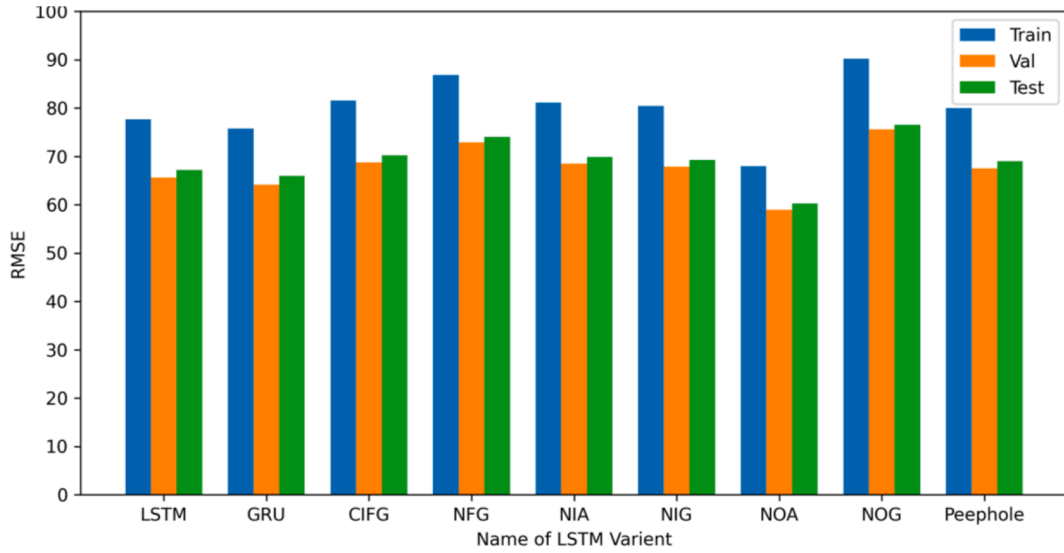


Fig. 13. RMSE on Energy Complete Dataset for Train data, Validation data, and Test data.

versions are measured in this paper. The vanilla LSTM employs complete gradient training and modifies the original LSTM (Hochreiter & Schmidhuber, 1997) as described in (Gers et al., 2000) and (Gers & Schmidhuber, 2000). Fig. 1 illustrates the internal representation of the LSTM Cell which also includes peephole connections. It has a single cell (also known as memory), input gate, output gate; forget gate, and peephole connections denoted by blue lines. The output of the LSTM cell is added back to the block input as well as all other gates. Considering

the total LSTM blocks in the LSTM network is N , $\mathbf{x}^t$ is the input vector at time t and M is the total observation in the dataset. Representation for the LSTM gates and their weights are as follows:

- Weights for bias: $b_z, b_i, b_f, b_o \in \mathbb{R}^N$
- Weight for Peephole: $p_i, p_f, p_o \in \mathbb{R}^N$
- Weights for Input: $W_z, W_i, W_f, W_o \in \mathbb{R}^{N \times M}$
- Weights for Recurrent: $R_z, R_i, R_f, R_o \in \mathbb{R}^{N \times M}$

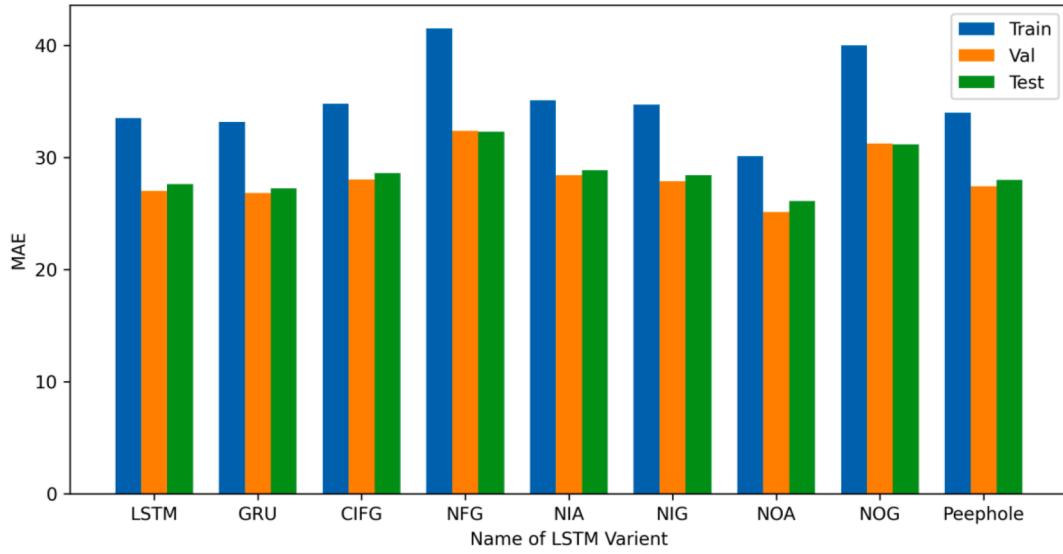


Fig. 14. MAE on Energy Complete Dataset for Train data, Validation data, and Test data.

Table 4

RMSE for train, validation, and testing dataset in sorted order for Energy appliances dataset.

Model Name	Training Data Set		Validation Data Set		Test Data Set	
	RMSE	MAE	RMSE	MAE	RMSE	MAE
NOA	67.941	30.125	58.937	25.153	60.258	26.127
GRU	75.794	33.191	64.198	26.847	65.968	27.253
LSTM	77.672	33.504	65.668	27.027	67.226	27.617
Peephole LSTM	79.974	34.001	67.496	27.445	69.044	28.017
NIG	80.438	34.722	67.858	27.896	69.234	28.400
CIFG	81.541	34.780	68.730	28.038	70.182	28.602
NIA	81.139	35.115	68.506	28.430	69.843	28.863
NOG	90.18	40.007	75.599	31.239	76.501	31.155
NFG	86.860	41.513	72.943	32.388	73.990	32.294

The forward pass of an LSTM can be calculated using the following equations:

$$z^t = \tanh(W_z X^t + R_z Y^{t-1} + b_z) \quad (1)$$

$$i^t = \sigma(W_i X^t + R_i Y^{t-1} + p_i \odot c^{t-1} + b_i) \quad (2)$$

$$f^t = \sigma(W_f X^t + R_f Y^{t-1} + p_f \odot c^{t-1} + b_f) \quad (3)$$

$$c^t = c^{t-1} \odot f^t + z^t \odot i^t \quad (4)$$

$$o^t = \sigma(W_o X^t + R_o Y^{t-1} + p_o \odot c^t + b_o) \quad (5)$$

$$y^t = \tanh(c^t) \odot o^t \quad (6)$$

In these equations, $\mathbf{h}$, σ , and $\mathbf{g}$ are non-linear activation functions. $\mathbf{g}(\mathbf{x})$ and $\mathbf{h}(\mathbf{x})$ are the hyperbolic tangents that are applied as the block output and output activation function. Sigmoid is used as an input gate activation function and forget gate activation function which is denoted by σ . Symbol $\odot$ denotes the point-wise multiplication of two vectors.

3. Methodology

The LSTM is composed of different gates to update and propagate information through the cell. Therefore, to assess the impact of individual components of the LSTM, those components needed to be isolated from the vanilla LSTM structure. This section describes all mutated versions of LSTM cells, which are created by removing exactly one

component sensibly from it and developing new LSTM cells. The derived variations from the base LSTM architecture are the following:

3.1. Base LSTM

This is probably the most used LSTM cell which was introduced by (Gers et al., 2000). The modification to the original LSTM (Hochreiter & Schmidhuber, 1997) was the introduction of a forget gate into the cell which was used to release resources by controlling what is to flow in the cell state. The mathematical expression of the base LSTM cell can be as follows and the pictorial connections of each component are shown in Fig. 2.

$$z^t = \tanh(W_z X^t + R_z Y^{t-1} + b_z) \quad (7)$$

$$i^t = \sigma(W_i X^t + R_i Y^{t-1} + b_i) \quad (8)$$

$$f^t = \sigma(W_f X^t + R_f Y^{t-1} + b_f) \quad (9)$$

$$c^t = c^{t-1} \odot f^t + z^t \odot i^t \quad (10)$$

$$o^t = \sigma(W_o X^t + R_o Y^{t-1} + b_o) \quad (11)$$

$$y^t = \tanh(c^t) \odot o^t \quad (12)$$

3.2. Peephole LSTM

To feedback precise time and count peephole connections were introduced in (Gers et al., n.d.). Peephole connections provide direct input from the cell state to all three gates of LSTM which essentially information about the importance of past events. As shown in Fig. 1 Blue dotted lines are the peephole connection at time $t-1$ and the solid line is the peephole connection at time t . The mathematical expressions are given in section 2. Peephole weights are indicated using P_i , P_f , and P_o which are for the input gate, forget gate, and output gate respectively.

3.3. LSTM without Forget Gate (No Forget Gate NFG)

This was the early LSTM cell (Hochreiter & Schmidhuber, 1997) to introduce a gating mechanism to the recurrent neural net. This can be achieved by setting a value of $f^t = 1$ in the base LSTM equations given in 3.4.1. This change will remove the effect of resetting information from the cell. Fig. 3 shows a diagram of the LSTM cell by removing the forget gate.

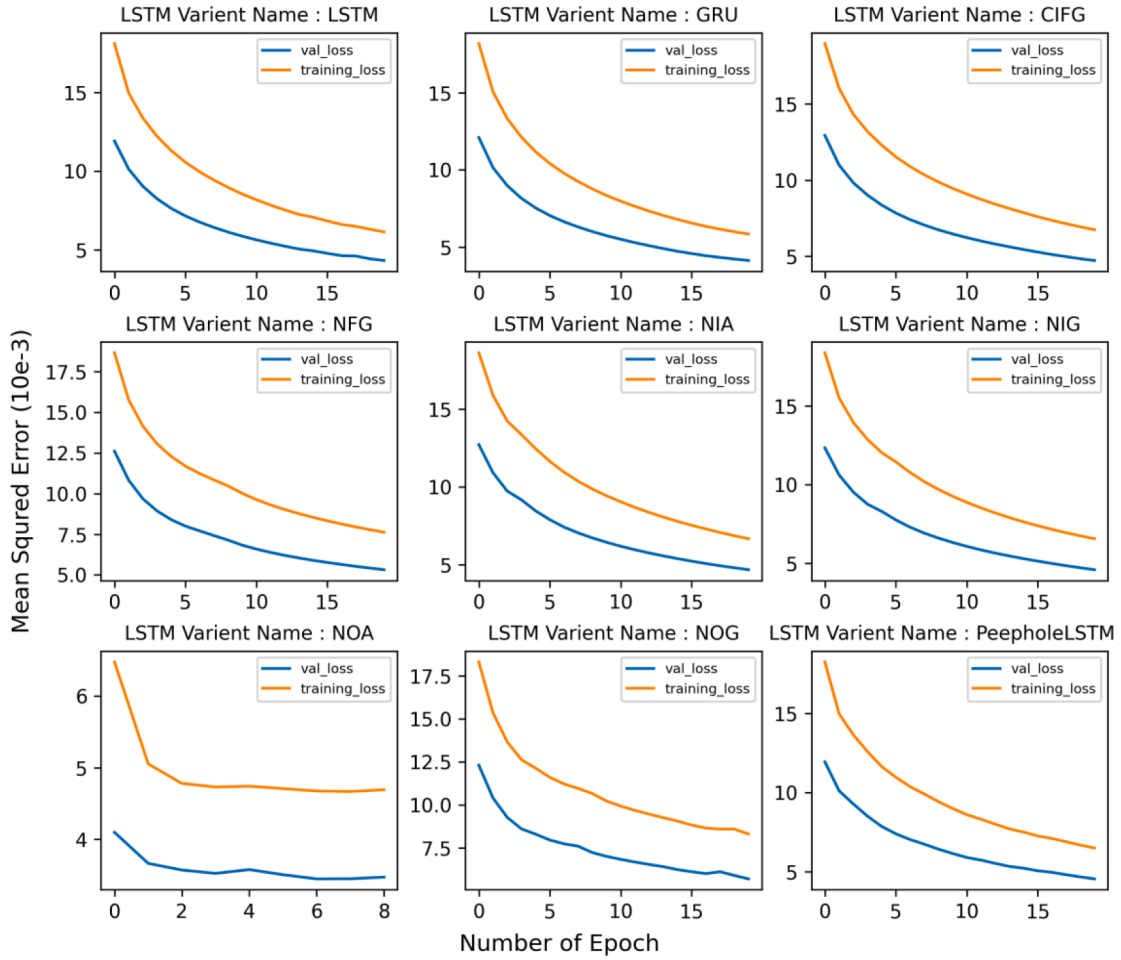


Fig. 15. Validation loss and training loss for each variant of the Energy appliances dataset.

3.4. LSTM without Input Gate (No Input Gate NIG)

In LSTM input gate is a mechanism to write selective data from the previous cell state and current input on the cell state. The mathematical expression of the input gate is given by equation 3.4.4 in base LSTM and the corresponding diagram is given in Fig. 4. Removal of the input gate can be achieved by setting $i^t = 1$.

3.5. LSTM without Output Gate (No Output Gate NOG)

Output gate controls information passed to the next time stamp in the sequence. Removing the output gate will remove the selective output of information and this will affect the output of each cell thereby affecting recurrent input to the network which can be achieved by setting $o^t = 1$ and this is shown in Fig. 5.

3.6. NIA No Input Activation Function

The input activation function is responsible for imputing the percentage of information in the cell memory. The input activation function is usually tanh and it is used to reduce the vanishing gradient in deep layers. LSTM after Removal of the activation function is depicted in Fig. 6 and the equation to achieve it can be given by modifying base LSTM equation no. 3 as $\sigma(i^t) = i^t$.

3.7. NOA No Output Activation Function

This change will control the state of the output gate as this will

control the total percentage of information out from the output gate which is represented in Fig. 7 and can be achieved by making $\sigma(o^t) = o^t$.

3.8. CIFG Coupled Input and Forget Gate

his modification combines the mechanism of input and forgets gate. Selective removal and addition of information on cell state is performed using the single mechanism of combined input and forget gate. We only forget when we're going to input something in its place. This is achieved by making the forget gate a function of the input gate as $f^t = 1 - i^t$ and is represented in Fig. 8.

4. Experimental setup

To evaluate the effect of each LSTM variant we have performed experiments on a single-step time-series forecasting task. In this section, we describe the datasets used for our experiments, present the network architecture, and outline the evaluation methods used to compare each variant of the LSTM.

4.1. Dataset details

Table 2 summarizes the characteristics of the dataset used for experiments. The first dataset is the Max Planck Institute for Biogeochemistry's Jena Climate dataset (Max-Planck-Institut Fuer Biogeochemie - Wetterdaten. (n.d.), 2022) which is a time series dataset with a recording of parameters such as temperature, and humidity at the Weather Station Jena, Germany by the Max Planck Institute for

Biogeochemistry. This dataset is frequently used to testify uni-variate and multi-variate time series forecasting methods. There are a total of 14 variables in the dataset which include temperature and humidity. The experiments in this paper use datasets from 2009 to 2016 and temperature time series. The original dataset is recorded every 10 min but it is resampled over 1 h time period using the assumption that temperature does not change significantly in 10 min.

The second dataset is about observations of household energy consumption. This data is augmented using temperature and humidity recorded using the ZigBee wireless sensor network every 10 min of the residential house and it is collected for about 4.5 months (Candanedo, 2017).

4.2. Data preprocessing

In the field of time series forecasting, uni-variate data refers to a single variable evolving. To effectively leverage the power of deep neural networks like Long Short-Term Memory (LSTM) for single-step forecasting, proper data pre-processing is essential. This process involves a series of steps to prepare the univariate time series data before feeding it into the LSTM model.

Check for missing values: Missing values can be caused by various reasons in time series but those values in time series have adverse effects on forecasting and it can put the model in an indecisive state. So, if there are missing values, it is recommended to handle them by either removing the corresponding rows or filling in the missing values using techniques like interpolation or forward/backward filling. In these experiments, we have used filling of missing values using the previous year's value by adding trends as both of these datasets are cyclical.

Normalize the data: Normalize the time series data to a common scale using Z-score normalization which is given by the formula:

$$X_{scaled} = \frac{(X - \text{Training_mean})}{\text{Training_std}} \quad (13)$$

Reshape the data: LSTM models expect input data in a specific format, usually a 3D array of shapes (samples, time steps, features). In the case of univariate time series data, samples are kept to 32 which is also called batch size. Lookback window for forecasting the next value which is set to 1 day which is equal to 24 timestamps.

Create input-output sequences: Convert the time series data into sequences of input-output pairs that can be fed into the LSTM model. Each input sequence corresponds to a window of past observations, and the output corresponds to the observation at the next timestamp. The window size is determined based on the problem and can be adjusted according to the specific requirements and for these experiments, the window size is set to 24.

Split input-output sequences into X and y: Separate the input sequences (X) and their corresponding output values (y). This step is necessary to distinguish between the input data and the target values during the training phase. Convert the sequences (X and y) into NumPy arrays, which are the preferred data format for most deep learning frameworks.

Divide data for training and testing: While dividing the pre-processed time series data typically, a large portion of the data is allocated for training. The remaining part is kept for testing and evaluating the model's performance. In these experiments, nearly 70 % of data is kept for training and 30 % of data is further divided into validation (10 %) and test dataset (20 %).

4.3. Network architecture and training

The network architecture of all our experiments is kept common for all LSTM cell variations as shown in Fig. 9. The primary aim of this study is to conduct an experimental comparison of various LSTM versions, rather than striving for groundbreaking results. In Fig. 9, the LSTM in the second layer serves as a representative placeholder for different LSTM

cell variations. The architecture in Fig. 9 is updated for each experiment with LSTM cell variations discussed in Section 3. To maintain a simple and equitable experimental setup, we tested eight different variations of the LSTM alongside the baseline LSTM. Each variant changes the baseline in exactly one way—removing or otherwise altering it—making it possible to isolate its impact.

The training process starts with data reading data. The inputs and outputs are independently retrieved from the training set and the test set, which are two divisions of the original time-series dataset. In these experiments mean squared loss is used for training along with Root mean squared error used as an error matrix. Adam optimizer is used for updating weights during back-propagation with 0.001 as the learning rate. All the models are evaluated maximum of 20 epochs as the objective is to compare on a common setup. Also, when there is no change in loss consecutively in two epochs training process will terminate assuming the model is not converging or the model's optimum weights and biases are then acquired.

5. Results & discussion

The findings from each experiment involving LSTM variants are presented in a table using various evaluation techniques. These experiments employ evaluation metrics such as the square root of the mean square error (RMSE) and the mean absolute error (MAE).

5.1. Evaluation Metrics

The performance of each experiment is evaluated based on the following evaluation matrix.

RMSE: Root-mean-square error (RMSE) is a frequently used measure in forecasting or regression to explain the differences between actual values and predicted values. It is given by the following formula:

$$RMSE = \frac{1}{n} \sqrt{\sum_{i=0}^n (y_i - \bar{y}_i)^2} \quad (14)$$

Where n is the total number of samples, y_i is the actual value and $\bar{y}_i$ is the predicted value.

RMSE is scale dependent which it cannot be used for comparison between two datasets. But it is good for describing positive or negative errors and it remains always greater than or equal to zero (zero practically cannot be achieved).

MAE: As the name it is the absolute difference between the observed value and estimated value using the model. This is given by the following formula:

$$MAE = \frac{1}{n} \sum_{i=0}^n |y_i - \bar{y}_i| \quad (15)$$

MAE along with RMSE increases confidence in the prediction accuracy as RMSE inflates error in some cases.

5.2. Results

In this section, we provide an analysis of the comparative results for the LSTM variations outlined in Section 3. The results of experiments conducted on the Jena Weather dataset for forecasting temperature are measured using MAE and RMSE which are depicted in Fig. 10 and Fig. 11 respectively. In Table 3, the error values of each variant are shown in ascending order. These experiments are also repeated on the Appliance energy usage dataset and those results are reported in Fig. 13 (for RMSE) and Fig. 14 (for MAE). Details of the best-performing model are given in Table 4 for RMSE and MAE.

In these experiments, the following key observations have been made:

- The GRU outperformed the LSTM on both datasets.
- NOA variation of LSTM performs best in the Appliance Energy Usage Dataset while it is second best on the Jena Weather dataset.
- The LSTM with peephole connections is not performing better compared to GRU or LSTM despite it having access to information in the memory at each gate.
- The forget gate of LSTM was considered the most important gate in (Mikolov et al., 2015). However, in this experiment, the removal of the forget gate yields mixed results. For instance, on the Jena weather dataset, the model without the forget gate (NFG) ranks third, whereas, on the Appliance energy usage dataset, it ranks last.
- Removal of the input gate or output gate results in worst performing variants of LSTM named NOG and NIG as those variations are in the last three places on both datasets. It indicates input gate and output gate mechanisms are very important gating mechanisms in searching for better-performing LSTM cells.
- Fig. 12 and Fig. 15 indicate all variations are converging for training and testing sets without overfitting or underfitting using experimental parameters discussed in section 4.

6. Conclusions

In this paper, we conduct experiments to identify the most efficient LSTM cell architecture, recognizing that LSTM will remain a fundamental technique for Time Series forecasting. LSTM cell architectures which are developed using mutations in the computational components resulted in the following new architectures: LSTM without output activation (NOA) or without input activation (NIA), LSTM without output gate (NOG) or forget gate (NFG) or input gate (NIG), Combined input and forget gate (CIFG), Gated recurrent unit (GRU), LSTM with peephole connection and most popular base LSTM. All the derived LSTM cell architectures are evaluated using identical hyper-parameters on two datasets—Jane Climate and Appliances Energy usage and results are compared based on RMSE and MAE (see Tables 3 and 4).

Experimental results indicate the superiority of the derived LSTM model (NOA) in terms of forecasting accuracy and generalization. These experiments also throw light on the importance of each gating mechanism of LSTM in time series forecasting. As per the results when the forget gate is removed, LSTM gives the worst performance while input gate removal has no significant change in result compared to base LSTM. In the future it would be worth investigating the capability of such mutated LSTM cell architecture on varied characteristics of time series, for example, the forecasting horizon. The hyperparameter tuning and finding of optimal network architecture using regularization techniques is also an important direction to experiment along with testing them on multivariate time series forecasting.

CRedit authorship contribution statement

Hemant Yadav: Conceptualization, Data curation, Methodology, Software, Visualization, Writing – original draft, Writing – review & editing. **Amit Thakkar:** Investigation, Validation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Alom, M. Z., Taha, T. M., Yakopcic, C., Westberg, S., Sidike, P., Nasrin, M. S., ... Asari, V. K. (2019). A state-of-the-art survey on deep learning theory and architectures. *Electronics*, 8(3), 292. <https://doi.org/10.3390/electronics8030292>
- Bandara, K., Bergmeir, C., & Smyl, S. (2020). *Forecasting Across Time Series Databases using Recurrent Neural Networks on Groups of Similar Series: A Clustering Approach*. 140. 10.1016/j.eswa.2019.112896.
- Candanedo, L. (2017). *Appliances energy prediction*. *UCI Machine Learning Repository*. 10.24432/C5VC8G.
- Chen, K., Zhou, Y., & Dai, F. (2015). A LSTM-based method for stock returns prediction: A case study of China stock market. *IEEE International Conference on Big Data (Big Data)*, 2015, 2823–2824. <https://doi.org/10.1109/BigData.2015.7364089>
- Cheng, D., Yang, F., Xiang, S., & Liu, J. (2022). Financial time series forecasting with multi-modality graph neural network. *Pattern Recognition*, 121, Article 108218. <https://doi.org/10.1016/j.patcog.2021.108218>
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation* (arXiv:1406.1078). arXiv. <http://arxiv.org/abs/1406.1078>.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling* (arXiv:1412.3555). arXiv. <http://arxiv.org/abs/1412.3555>.
- Dash, S., Chakraborty, C., Giri, S. K., & Pani, S. K. (2021). Intelligent computing on time-series data analysis and prediction of COVID-19 pandemics. *Pattern Recognition Letters*, 151, 69–75. <https://doi.org/10.1016/j.patrec.2021.07.027>
- Deb, C., Zhang, F., Yang, J., Lee, S. E., & Shah, K. W. (2017). A review on time series forecasting techniques for building energy consumption. *Renewable and Sustainable Energy Reviews*, 74, 902–924. <https://doi.org/10.1016/j.rser.2017.02.085>
- Ensaifi, Y., Amin, S. H., Zhang, G., & Shah, B. (2022). Time-series forecasting of seasonal items sales using machine learning – A comparative analysis. *International Journal of Information Management Data Insights*, 2(1), Article 100058. <https://doi.org/10.1016/j.ijimei.2022.100058>
- Fernández-Martínez, D., & Jaramillo-Morán, M. A. (2022). Multi-Step Hourly Power Consumption Forecasting in a Healthcare Building with Recurrent Neural Networks and Empirical Model Decomposition. *Sensors*, 22(10), 3664. <https://doi.org/10.3390/s22103664>
- Gers, F. A., & Schmidhuber, J. (2000). Recurrent nets that time and count. *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, 189–194 vol.3. 10.1109/IJCNN.2000.861302.
- Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, 12(10), 2451–2471. <https://doi.org/10.1162/089976600300015015>
- Gers, F. A., Schraudolph, N. N., & Schmidhuber, J. (n.d.). *Learning Precise Timing with LSTM Recurrent Networks*.
- Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM networks. *Proceedings. 2005 IEEE International Joint Conference on Neural Networks*, 2005., 4, 2047–2052. 10.1109/IJCNN.2005.1556215.
- Hewage, P., Trovati, M., Pereira, E., & Behera, A. (2021). Deep learning-based effective fine-grained weather forecasting model. *Pattern Analysis and Applications*, 24(1), 343–366. <https://doi.org/10.1007/s10044-020-00898-1>
- Hewamalage, H., Bergmeir, C., & Bandara, K. (2021). Recurrent Neural Networks for Time Series Forecasting: Current Status and Future Directions. *International Journal of Forecasting*, 37(1), 388–427. <https://doi.org/10.1016/j.ijforecast.2020.06.008>
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Lara-Benítez, P., Carranza-García, M., García-Gutiérrez, J., & Riquelme, J. C. (2020). Asynchronous dual-pipeline deep learning framework for online data stream classification. *Integrated Computer-Aided Engineering*, 27(2), 101–119. <https://doi.org/10.3233/ICA-200617>
- Lim, S.-C., Huh, J.-H., Hong, S.-H., Park, C.-Y., & Kim, J.-C. (2022). Solar Power Forecasting Using CNN-LSTM Hybrid Model. *Energies*, 15(21), 8233. <https://doi.org/10.3390/en15218233>
- Masini, R. P., Medeiros, M. C., & Mendes, E. F. (2021). *Machine Learning Advances for Time Series Forecasting* (arXiv:2012.12802). arXiv. <http://arxiv.org/abs/2012.12802>.
- Max-Planck-Institut fuer Biogeochemie—Wetterdaten. (N.d.). (2022).
- Mikolov, T., Joulin, A., Chopra, S., Mathieu, M., & Ranzato, M. (2015). *Learning Longer Memory in Recurrent Neural Networks* (arXiv:1412.7753). arXiv. <http://arxiv.org/abs/1412.7753>.
- Ng, K. W., Horawalavithana, S., & Iamnitchi, A. (2022). Social media activity forecasting with exogenous and endogenous signals. *Social Network Analysis and Mining*, 12(1), 102. <https://doi.org/10.1007/s13278-022-00927-3>
- Nielsen, M. (2015). *Neural Networks and Deep Learning* (Vol. 25).
- Ömer Faruk, D. (2010). A hybrid neural network and ARIMA model for water quality time series prediction. *Engineering Applications of Artificial Intelligence*, 23(4), 586–594. <https://doi.org/10.1016/j.engappai.2009.09.015>
- Pan, C., Tan, J., Feng, D., & Li, Y. (2019). Very Short-Term Solar Generation Forecasting Based on LSTM with Temporal Attention Mechanism. *2019 IEEE 5th International Conference on Computer and Communications (ICCC)*, 267–271. 10.1109/ICCC47050.2019.9064298.
- Rangapuram, S. S., Seeger, M., Gasthaus, J., Stella, L., Wang, Y., & Januschowski, T. (n.d.). *Deep State Space Models for Time Series Forecasting*.
- Reyes, O., & Ventura, S. (2019). Performing Multi-Target Regression via a Parameter Sharing-Based Deep Network. *International Journal of Neural Systems*, 29(09), 1950014. <https://doi.org/10.1142/S012906571950014X>

- Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3), 1181–1191.
- Shewalkar, A., Nyavanandi, D., & Ludwig, S. A. (2019). Performance Evaluation of Deep Neural Networks Applied to Speech Recognition: RNN, LSTM and GRU. *Journal of Artificial Intelligence and Soft Computing Research*, 9(4), 235–245. <https://doi.org/10.2478/jaiscr-2019-0006>
- Tian, Y., & Pan, L. (2015). Predicting Short-Term Traffic Flow by Long Short-Term Memory Recurrent Neural Network. *IEEE International Conference on Smart City/SocialCom/SustainCom (SmartCity)*, 2015, 153–158. <https://doi.org/10.1109/SmartCity.2015.63>
- Türkmen, A. C., Januschowski, T., Wang, Y., & Cemgil, A. T. (2021). Forecasting intermittent and sparse time series: A unified probabilistic framework via deep renewal processes. *PLOS ONE*, 16(11), e0259764.
- Van Houdt, G., Mosquera, C., & Nápoles, G. (2020). A review on the long short-term memory model. *Artificial Intelligence Review*, 53(8), 5929–5955. <https://doi.org/10.1007/s10462-020-09838-1>
- Wang, Y., Smola, A., Maddix, D. C., Gasthaus, J., Foster, D., & Januschowski, T. (n.d.). *Deep Factors for Forecasting*.
- Yu, J. J. Q. (2022). Graph Construction for Traffic Prediction: A Data-Driven Approach. *IEEE Transactions on Intelligent Transportation Systems*, 23(9), 15015–15027. <https://doi.org/10.1109/TITS.2021.3136161>
- Yu, Y., Si, X., Hu, C., & Zhang, J. (2019). A Review of Recurrent Neural Networks: LSTM Cells and Network Architectures. *Neural Computation*, 31(7), 1235–1270. https://doi.org/10.1162/neco_a_01199
- Zhao, Y., Ye, L., Li, Z., Song, X., Lang, Y., & Su, J. (2016). A novel bidirectional mechanism based on time series model for wind power forecasting. *Applied Energy*, 177, 793–803. <https://doi.org/10.1016/j.apenergy.2016.03.096>